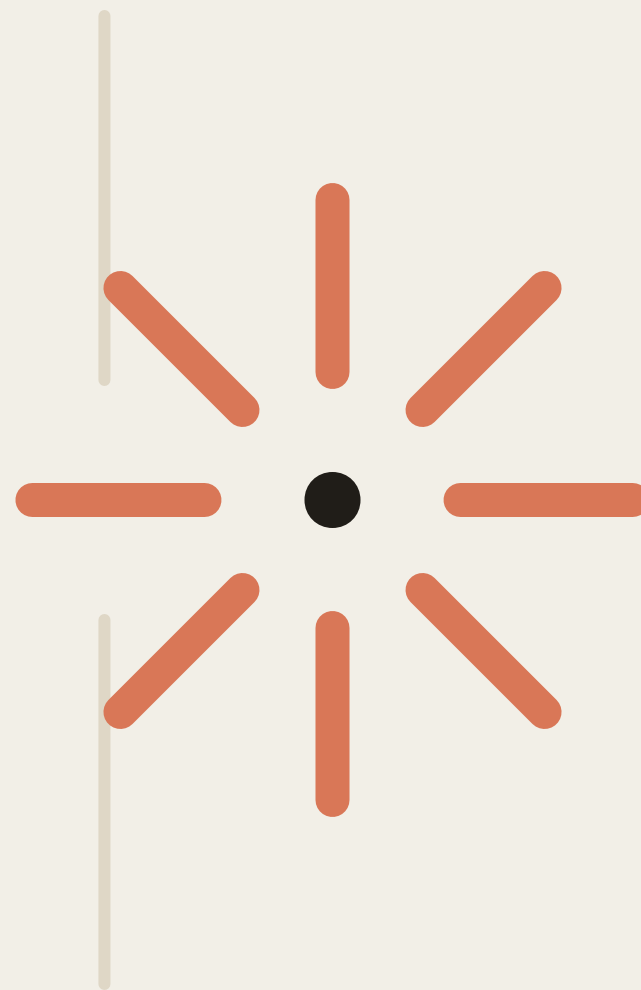


One file. One decision.

Same spec, same eight-word prompt. We add sixty-nine lines to `.claude/` and nothing else. Whatever changes in the review is attributable to those sixty-nine lines.

> Read SPEC.md and build it.



Ten findings, in two very different piles.

PILE ONE • ARCHITECTURE

The spec never said how the UI reaches its data, so the agent picked. Roughly five findings, and every one of them traces back to **a single decision** the document declined to make.

An unanswered question, not sloppiness.

PILE TWO • QUALITY

A swallowed exception, a suppressed warning, a skipped guard, a component doing four jobs, a status compared as a string. Roughly five more.

Nothing ambiguous about these. A careful reviewer rejects them.

WHY THE TWO PILES MATTER

Two piles, two different fixes. Pile one is a missing decision, fixed by making the decision somewhere durable. Pile two is a missing standard, fixed by writing the standard down.

Today removes pile one. Checkpoint 4 removes pile two. Doing them separately is what proves each one worked.

IF YOUR NUMBERS DIFFERED

They will, a little, because a live build is not deterministic. The **shape** is what repeats: a cluster of findings around one architectural choice, plus a handful of independent quality slips.

Check the diff yourself. It is one file.

```
git diff checkpoint-1 checkpoint-2
```

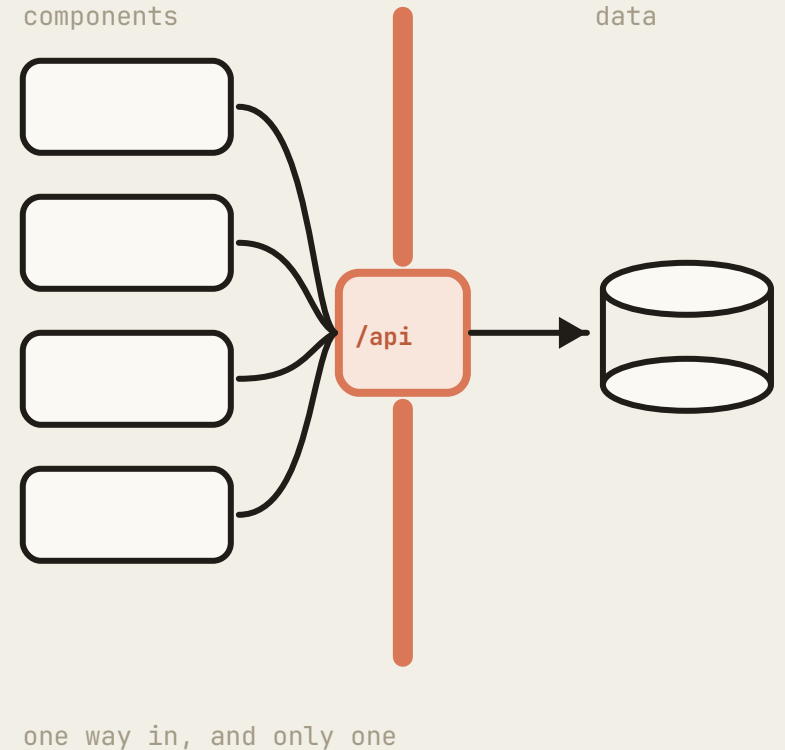
```
+ .claude/skills/api-boundary/SKILL.md 69 lines
• SPEC.md unchanged, byte for byte identical
• .claude/skills/review-build/SKILL.md unchanged
• .claude/agents/assetdesk-reviewer.md unchanged
• the prompt you type unchanged
```

The reviewer is unchanged too, and that matters more than it looks: it reads **whatever skills the branch happens to ship**. Adding the skill changes how the code gets written, and also what counts as a violation. Same auditor, wider rulebook.

`git diff checkpoint-1 checkpoint-2 --stat`. Run it before you build, so you believe the claim.

A standard that shows up without being asked.

You are not going to remember to paste "the UI must go through the API" into every session, and neither is the colleague who joins in March. A skill is that sentence, committed to the repo, with a **description that tells Claude when it applies**, so it loads itself on the turns where it matters.



- the standard lives in the head of whoever reviews the pull request
- + the standard lives in `.claude/skills/` and arrives before the code is written

Three places to put a rule. They are not interchangeable.

	SPEC.MD	A SKILL	A PROMPT
Answers	What are we building	How do we build anything	What do I need right now
Lifetime	This product	Every product your team writes	This turn
Loads	When you point at it	On its own, when relevant	When you type it
Travels to the next repo	No	Yes, copy the folder	No
Binding on the reviewer	Yes	Yes	No, it never sees it

A rule you only ever said out loud is **invisible to every audit**. The reviewer runs in a fresh window and reads files, not your chat history.

The description is the trigger.

```
.claude/skills/api-boundary/SKILL.md
---
name: api-boundary
description: Use when building or changing
  AssetDesk UI or data access - requires the UI to
  reach data only through the HTTP API and never
  through AssetRepository directly.
---

# The UI talks to the API, never to the repository

AssetDesk keeps one boundary: components speak
HTTP, the repository speaks SQL, and
nothing crosses.
```

WHY THAT DESCRIPTION IS WRITTEN LIKE THAT

It names the **trigger** ("building or changing UI or data access") and then the **ruling**. Claude reads descriptions to decide what to load. "Best practices for our app" would never fire, because it matches nothing in particular.

FOLDER NAME = COMMAND NAME

skills/api-boundary/ also gives you /api-boundary if you ever want to invoke it by hand. Today you will not need to; it loads itself.

READ IT BEFORE YOU RUN THE PROMPT

Sixty-nine lines, two minutes. You are about to watch a build obey it, and the reviewer is about to score against it. Both are more interesting if you know what it says.

Each one is checkable with a grep.

1 · NO COMPONENT TOUCHES THE REPOSITORY

Not `@inject`, not `[Inject]`, not a parameter type, not a `using`. It is constructed in `Program.cs` and reachable only from the endpoint handlers.

2 · EVERY READ AND WRITE GOES THROUGH /API/*

The six endpoints the spec already required. They stop being test scaffolding and become the actual data path.

3 · ONE TYPED CLIENT, REGISTERED IN DI

`AssetDeskClient` wraps `HttpClient`. Never new `HttpClient()` at a call site: it bypasses your configuration and exhausts sockets under reconnects.

4 · EVERY CALL TAKES A CANCELLATION TOKEN

And checks the status code before deserialising. A Blazor circuit that drops mid-request must not leave the call running.

5 · DTOS AT THE BOUNDARY

The records in `Models.cs` are the database's shape. Components take DTOs from `Data/Dtos.cs`. A domain record must never appear in a `[Parameter]`.

6 · TRANSLATE ERRORS ONCE, IN THE CLIENT

A 400 carries `{"error": "..."}.` The client turns it into an `AssetDeskException` carrying that exact message. Components render it verbatim and never write their own copy. **SPEC.md §4.6 already demanded that**, and this is what makes it true across HTTP.

In rule 6 the skill is not overriding the spec. It is **keeping a promise the spec made** across a boundary the spec never mentioned.

This boundary is **not free**.

WHAT YOU PAY

Handlers become async. HTTP has no synchronous escape that does not risk deadlock, so every event handler turns into `async Task`.

A second shape to maintain. Every model gets a DTO twin, and both change together.

An extra hop. The server calls its own endpoint over the loopback for data it already has in memory.

THE SKILL SAYS SO OUT LOUD

It even reconciles itself with the spec: §3.4 still requires **synchronous Dapper inside the repository**. That rule is about the data layer's own calls, not the UI's HTTP hop. Both are correct at once.

THE TRAP IT HAD TO DOCUMENT

The app calls itself, so the base address is not known until the server is listening. Hardcode `localhost:5198` and it breaks the moment anyone passes a different `--urls`, and the scaffold writes a **random** port into `launchSettings`. The skill resolves it from `IServerAddressesFeature` instead.

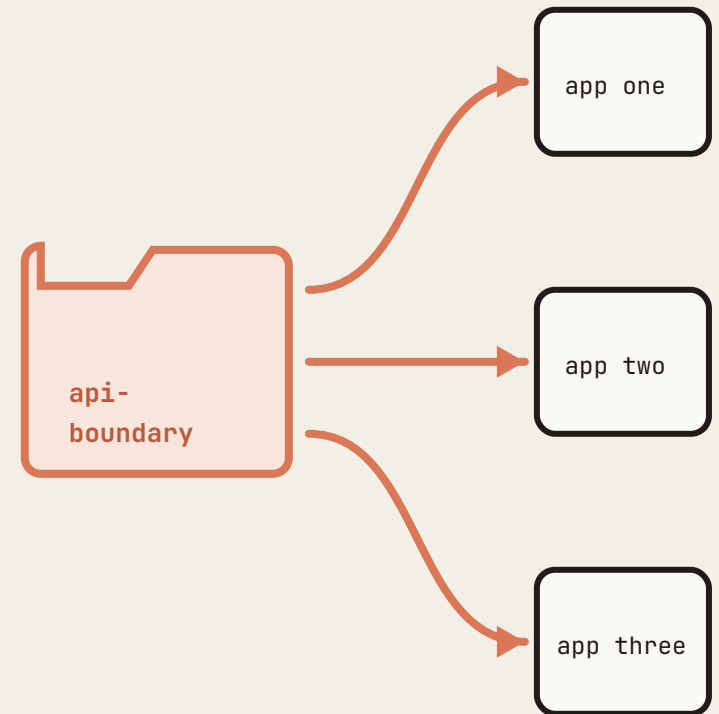
AND ONE EXPECTED LOG LINE

Failed to determine the https port for redirect is **normal** here and not a warning to chase. Only an http address is bound, so redirection is skipped and the self-call works.

A skill that only lists benefits is marketing. This one states its cost, its interaction with the spec, and the one thing that will bite you.

Product facts in the spec. Standards in a **skill**.

"Assets have a unique tag" is a fact about AssetDesk: it belongs in the spec and dies with the product. "The UI reaches data through an API" is a fact about **how your team builds anything**. Put it in one app's spec and you will rewrite it in the next app's spec, slightly differently, and nobody will notice they diverged.



written once, enforced everywhere

- paragraph 3.4b of a 600 line document nobody re-reads on project two
- + a folder you copy into the next repo, with its own history

One bucket empties. The other **does not move**.

CHECKPOINT 1

BOUNDARY: ~5
QUALITY: ~5
SPEC: ~0
TOTAL: ~10



CHECKPOINT 2 • PREDICTED

BOUNDARY: 0
QUALITY: ~5
SPEC: ~0
TOTAL: ~5

WHY THIS IS WORTH SAYING FIRST

A prediction you make before running the command can be **wrong**. That is what makes the result mean something, instead of a story assembled afterwards to fit whatever appeared.

IF QUALITY ALSO DROPS

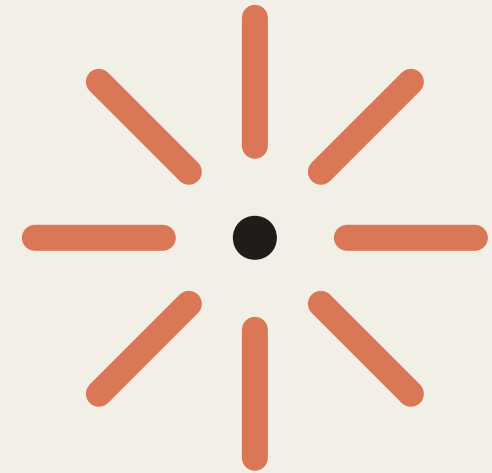
Then something other than the skill moved, and we should find out what. Say so out loud rather than taking the win.

Nothing in `api-boundary` mentions swallowed exceptions, suppressed warnings, or magic strings. It should not fix them, and if it appears to, be suspicious.

Four greps settle it **before** the reviewer does.

```
# rule 1 · must print nothing at all
grep -rn AssetRepository AssetDesk/Components/
# rule 3 · exactly one registration, zero ad hoc clients
grep -c 'AddHttpClient<AssetDeskClient>' AssetDesk/Program.cs
grep -rn 'new HttpClient' AssetDesk/
# rule 5 · the DTO file exists
ls AssetDesk/Data/Dtos.cs
```

/review-build



then compare the count block
checkpoint 3 →